

Project Jarvis: A Production-Ready Multi-Agent AI Architecture for Shumaker Roofing Co. (Marketing, Sales & Reporting)

TL;DR

- **Build it as Claude Agent SDK (orchestrator + subagents) running on a cloud VPS (Hetzner/DigitalOcean), connecting to every platform through read-scoped MCP servers.** This combination gives the lowest-friction path to a 24/7, auditable, hub-and-spoke system. The Mac Mini is only justified if iMessage delivery or local browser/screen automation is a hard requirement.
- **The AccuLynx “dealbreaker” is FALSE.** AccuLynx publishes a documented public REST API v2 (`https://api.acculynx.com/api/v2` , Bearer API-key auth) with read endpoints for Jobs, Milestones, Financials, Invoices, Payments, Sales Owner, and Users (reps) — everything the Sales sub-agent needs. The real constraints are per-location API keys, a 10 req/sec/key rate limit, and write-capable keys that must be locked down.
- **Start with a 2-agent split (combined Marketing+Sales + a separate Reporting/SQL agent), not 3.** Marketing and Sales share the same attribution spine (GCLID/UTM → lead → job), so fusing them early reduces routing overhead; the read-only Postgres agent should be isolated from day one because it carries the highest blast radius.

Key Findings

1. **Connector feasibility is far better than the brief assumed.** All five target systems expose usable developer APIs. AccuLynx (the flagged risk) has a full OpenAPI-documented v2 with an `llms.txt` index built for AI agents. Google Ads, GoHighLevel v2, and CallRail v3 are all mature, well-documented REST APIs.
2. **Authentication is heterogeneous and must be abstracted.** You will simultaneously manage OAuth2 three-legged flows with 24-hour token rotation (Google Ads, GHL v2) and static Bearer/API-key auth (AccuLynx, CallRail). This argues strongly for wrapping each platform in its own MCP server that owns credential lifecycle.
3. **Read-only is enforceable but not free.** Several APIs issue keys that are write-capable by default — AccuLynx states verbatim: *“Your API Key can be used by anyone to expose or manipulate your company data. Treat your API key like a password and don’t share it.”* There is no read-only key scope, so read-only must be enforced at the database role, the tool-allowlist, and the prompt-guardrail layers — defense in depth.
4. **The Claude Agent SDK natively implements the hub-and-spoke topology** via subagents with isolated context windows, per-subagent `allowedTools` , and the `mcp__<server>__<tool>` permission pattern — eliminating most custom orchestration glue code.
5. **Voice is a bolt-on, not a foundation.** The clean path is to keep Jarvis’s reasoning core channel-agnostic and add voice later via Vapi (orchestration) + ElevenLabs (TTS) over WebSockets/WebRTC, or Pipecat if self-hosting.

Details

Layer 1 — Host Infrastructure

Option	Strengths	Weaknesses	Verdict
Cloud VPS (Hetzner/DigitalOcean)	No hardware dependency; trivial remote access (SSH/CLI/WebSockets); snapshots + instant resize; systemd auto-restart on reboot; DigitalOcean has the largest app marketplace and one-click agent images; ~\$6–80/mo	No native macOS/iMessage; you manage OS hardening	Recommended primary host for a read-only, API-driven marketing/sales/reporting agent that never needs a local GUI

Dedicated Mac Mini (M4)	Only path to native iMessage (via BlueBubbles REST bridge) and local screen/browser automation that needs a real WindowServer; unified-memory efficiency for any local LLM fallback; ~\$599 once + ~\$15/yr power; break-even vs VPS in 8–20 months	CapEx; home-network/ISP single point of failure; macOS updates force reboots; headless requires an HDMI dummy plug to keep WindowServer alive	Recommended only if iMessage or local browser/screen automation is mandatory
Replit (Reserved VM)	Fastest to prototype; integrated secrets; "Background worker" deploy type for non-HTTP agents	Reserved VM is the only tier that is genuinely always-on (Autoscale sleeps; Scheduled Deployments cap at 11h); costs creep with compute/bandwidth; least control over OS-level browser drivers	Prototype/MVP only , migrate off for production

Recommendation: Host the production orchestrator on a **Hetzner or DigitalOcean VPS** (4–8 GB RAM is ample for an API-orchestration agent calling a hosted frontier model). Reserve a Mac Mini exclusively for an iMessage bridge if the principals insist on texting Jarvis from their iPhones — and even then, run the bridge on the Mac while keeping the brain on the VPS, accepting the documented downside of split state across two machines.

For 24/7 reliability regardless of host: (1) run the agent under **systemd** (Linux) or **launchd** with `KeepAlive:true` (macOS) for sub-5-second crash restart; (2) mount persistent storage so memory/session state survives reboots; (3) disable all sleep; (4) use **Tailscale** for zero-config encrypted remote admin.

Layer 2 — Agentic Harness & Core Framework

Framework	Persistent memory	Vector/knowledge injection	Inter-agent loops	Fit for Jarvis
Claude Agent SDK (Claude Code SDK mode) + MCP	Session persistence + <code>resume / forkSession</code> ; CLAUDE.md project memory; automatic context compaction	Skills (progressive disclosure) + MCP-exposed vector stores	First-class subagents with isolated context, per-agent tools/permissions, <code>Task</code> tool delegation	Top pick — native hub-and-spoke, native MCP, least glue code
LangGraph	Best-in-class durable checkpointing; resumable graphs; survives process crashes	BYO vector store via LangChain	Explicit cyclic state graph; <code>interrupt()</code> for human-in-loop	Strong runner-up — choose if you need auditable, deterministic cyclic workflows or model-agnosticism
CrewAI	Role-based memory + RAG; weaker logging	Built-in RAG	Intuitive role/crew delegation but sequential process model struggles with cycles	Good for rapid prototype; migrate out for complex state
AutoGen	Conversation history; needs custom persistence	BYO	Conversational/group-chat actor model	Best for agent-debate use cases; overkill here
Hermes Agent (Nous Research)	Three-layer memory (episodic/semantic/procedural); SQLite FTS5 recall; auto-generates SKILL.md; 20 messaging channels out of the	Markdown memory files + pluggable vector providers (Mem0, Supermemory)	Single-gateway multi-channel; emerging sub-agent support	Compelling for the channel layer (it natively bridges Slack/iMessage/Telegram with persistent cross-platform memory) but CLI-first, MIT, and its self-improving-memory surface is a prompt-injection

	box			risk for a system touching financial data
--	-----	--	--	---

Recommendation: Claude Agent SDK as the harness, because the orchestrator→sub-agent topology in Layer 3 maps 1:1 onto its subagent model, and every connector in Layer 4 plugs in as an MCP server with a wildcard `allowedTools` scope. The single biggest trade-off to accept: the SDK locks you to Claude models and runs by spawning a CLI subprocess (a deployment nuance, not a blocker). If model-agnosticism or strict deterministic auditability becomes a hard requirement, **LangGraph is the fallback**. Hermes is best considered as an optional channel-gateway shell, not the reasoning core.

Layer 3 — Orchestrator & Sub-Agent Topography

Topology: Hub-and-spoke. Jarvis (orchestrator) owns query parsing, routing, and synthesis; sub-agents own a single domain and a single credential set.

- **Orchestrator (Jarvis):** parses the natural-language query, decomposes it into per-domain sub-questions, dispatches to sub-agents in parallel (`Task` tool), then synthesizes a single answer. Holds no platform credentials itself.
- **Sub-Agent A — Marketing:** Google Ads (campaign/PMAX performance, ROAS), GHL v2 (funnel stages, lead attribution), CallRail (source/medium/GCLID first-touch). Tools: `mcp_googleads__*`, `mcp_ghl__*`, `mcp_callrail__*` (read-scoped).
- **Sub-Agent B — Sales:** AccuLynx (pipeline milestones, per-rep sales-owner data, financial totals), CallRail (lead qualification / `lead_status`, call handling). Tools: `mcp_acculynx__*`, `mcp_callrail__*` (read-scoped).
- **Sub-Agent C — Reporting:** read-only PostgreSQL ROI dashboard. Tools: `mcp_postgres_query` only.

2-agent vs 3-agent recommendation: Start combined Marketing+Sales (one agent) + a separate Reporting agent.
Rationale: the marketing and sales domains share the attribution spine — a GCLID from Google Ads becomes a CallRail call, becomes a GHL opportunity, becomes an AccuLynx job — so a single agent that holds both contexts answers cross-funnel questions (“which campaigns produce qualified AccuLynx jobs?”) without an extra orchestration hop. The Reporting/SQL agent is split out immediately because it has the highest blast radius (raw DB access) and the most distinct guardrail profile. **Promote to the full 3-agent split when** (a) the combined agent’s tool list exceeds ~20–25 tools and routing accuracy degrades, or (b) Marketing and Sales develop genuinely independent escalation/alerting cadences.

Layer 4 — Connectors & API Feasibility (CRITICAL)

AccuLynx — the flagged “dealbreaker,” investigated thoroughly

Verdict: Not a dealbreaker. AccuLynx has a documented public REST API v2.

- **Base URL / auth:** `https://api.acculynx.com/api/v2`, `Authorization: Bearer <API_KEY>`. (dltHub) Keys are generated in the AccuLynx API settings by a company administrator; **each company location requires its own key**.
- **Read endpoints relevant to the Sales sub-agent (all confirmed in the public OpenAPI/ 11ms.txt index):** `GET /jobs` (filterable by date range, milestone, sort) and `GET /jobs/search`; `GET /jobs/{id}/milestones` and milestone history; `GET /jobs/{id}/financials` (supports worksheet + amendments), `GET /jobs/{id}/invoices`, `GET /jobs/{id}/payments`, and `GET /jobs/{id}/payments-overview` (high-level financial overview); (acculynx) `GET /jobs/{id}/sales-owner` (the rep on a job) and `GET /jobs/{id}/representatives`; `GET /users` (status-filterable list of reps); (acculynx) active lead sources, trade/work types, custom fields.
- **Webhooks:** A full webhook subscription API exists (topics include `job-milestone-changed`, `approvedjobvaluechanged`, `invoiceupdated`) — preferred over polling for change-driven updates.
- **Rate limits (confirmed verbatim from AccuLynx docs):** “Every application is subject to an IP-based concurrent request limit of 30 requests per second. If a single IP is making more than 30 requests per second, new requests will be denied and return an HTTP 429 status code” — then 10 requests/sec per API key. Higher limits require applying through the **AccuLynx Partner Program**.
- **REAL blockers to flag aggressively: (1) Zero developer support** — AccuLynx states verbatim: “We do not support our APIs in the same way we support our AccuLynx Pro, Elite, or mobile apps. We do not coach or teach people how to use our

APIs, nor do we explain how to use different tools to connect to our APIs.” You are on your own, with only a Contact Us form for troubleshooting. (2) **Per-location keys** mean a multi-location Shumaker footprint multiplies key management. (3) **Keys are write-capable** — AccuLynx warns “Your API Key can be used by anyone to expose or manipulate your company data” — there is no read-only key scope, so read-only must be enforced in the MCP tool layer by only implementing GET tools. (4) AccuLynx published **no detailed pagination/back-pressure guidance beyond the above** — implement exponential backoff with jitter.

Feasibility Matrix

Platform	API openness	Auth	Key read endpoints	Rate limits	Known roadblocks
Google Ads API	Mature, fully public (v24+)	OAuth2 (3-legged or service account) + developer token	<code>GoogleAdsService.SearchStream</code> w/ GAQL: <code>campaign, metrics.cost_micros, metrics.conversions_value</code> (ROAS), PMax via <code>asset_group</code>	10k ops/batch; access-level quotas (Basic = 15,000 ops/day)	Developer token requires Google approval (Basic→Standard); token cannot hit production accounts until approved; cost in <code>micros</code> (÷1e6)
GoHighLevel v2	Mature, public	OAuth2 (Authorization Code) or Private Integration Token; access token expires ~24h, refresh token ~1yr Gohighlevel	Opportunities/pipelines (funnel stages), Contacts (lead attribution), Conversations	100 req/10s burst; 200,000/day Ghllogic per app per location	V1 end-of-support 31-Dec-2025 — must use v2; OAuth advanced features gated to Agency Pro plan; <code>companies.readonly</code> scope available
AccuLynx	Public REST v2 (documented)	Bearer API key (per location)	Jobs, Milestones, Financials, Invoices, Payments, Sales Owner, Users	30 req/s per IP; 10 req/s per key	No dev support; per-location keys; keys are write-capable (no read-only scope); partner program for higher limits
CallRail v3	Mature, public	<code>Authorization: Token token=<API_KEY></code> callrail	<code>/calls.json</code> w/ <code>lead_status, value, tags, keywords</code> ; Premium CI adds <code>transcription, sentiment, lead_score</code>	1,000/hr, 10,000/day (general)	Paid API usage tiers (see below); transcripts require Premium Conversation Intelligence plan; 25-month data retention
PostgreSQL	N/A (direct)	DB role credentials	Read-only <code>SELECT</code> on ROI dashboard	DB-configured	Text-to-SQL injection/write risk — see Layer 5

Note on Google Ads token approval (corrected critical-path estimate): Google’s official targets are roughly two business days for a Basic Access review and up to ~10 business days for Standard Access. However, as of early 2026 Google acknowledged a backlog and longer review times on developer-token applications. **Plan for two business days as the optimistic case but weeks in practice** — this is the single longest external dependency on the build, so submit the application before anything else.

CallRail lead-qualification specifics (Sales sub-agent)

- `lead_status` field — values `good_lead, not_a_lead, previously_marked_good_lead, or null`; callrail also a filter and sort key. This is the primary lead-qualification signal.

- `value / formatted_value` — monetary value assigned to a call.
- `tags`, `keywords` (paid-source search terms), `milestones.qualified` object (*“the last source before a customer becomes scored as a qualified lead”*), `person_id` (lead identity across calls).
- **Premium Conversation Intelligence add-on** unlocks `transcription`, `conversational_transcript`, `sentiment`, `call_summary`, `call_highlights`, `lead_score` (AI grade “very poor”→“very good”) `callrail` and `lead_score_explanation`.
- **Most qualification fields are NOT in the default response** — they must be explicitly requested via the `fields=` query parameter.
- **API pricing tiers (CallRail charges for API usage, confirmed from CallRail Help Center, updated June 2, 2026):** Free = 0–1,500 API calls/mo (no charge); **Premium = 1,501–12,000/mo = \$80 USD/mo**; **Platinum = 12,001–300,000/mo = \$200 USD/mo**. Calls made through CallRail's native Google/Meta/HubSpot integrations do not count toward usage. This is a real recurring cost to budget, separate from the product subscription.

PostgreSQL — secure read-only text-to-SQL

Map the ROI dashboard DB into the Reporting agent through a Postgres MCP server that exposes a **single read-only query tool**, backed by a database role created with `GRANT SELECT` only (no INSERT/UPDATE/DELETE/DDL) and ideally pointed at a **read replica**. See Layer 5 for the full guardrail stack.

Layer 5 — Communication Channels

- **Slack (primary):** Use **Bolt** (official SDK; JS/Python/Java). Run in **Socket Mode** so the agent needs no public inbound URL (ideal on a hardened VPS) — requires an app-level token (`xapp-`) with `connections.write` plus the bot token (`xoxb-`). Verify the Slack signing secret on every request. Apply least-privilege scopes.
- **iMessage (secondary, optional):** Requires macOS. The robust path is **BlueBubbles Server** (runs on the Mac, exposes a REST API with reactions/edits/media; needs Full Disk Access + a Firebase project for push). This is the single strongest reason to introduce a Mac Mini. Note Apple ToS risk around automated messaging.
- **Voice (future bolt-on):** Keep Jarvis's reasoning core channel-agnostic so voice attaches at the transport edge, not the brain. Recommended later stack: **Vapi** for full-pipeline orchestration (STT↔LLM↔TTS, barge-in, sub-second latency over WebRTC) with **ElevenLabs** as the TTS voice; or **Pipecat** (open-source, WebSocket/WebRTC transports, Twilio serializer for telephony) if self-hosting the pipeline. Vapi's BYO-model approach lets Jarvis remain the LLM brain while Vapi manages the audio plumbing. Budget ~\$0.23–0.33/min all-in for a BYOK Vapi stack.

Layer 6 — Skills, Vector Stores & Knowledge Injection

Goal: sub-agents must never hallucinate business logic (pricing, the definition of a “qualified lead,” company terminology).

- **Author business logic as Markdown “Skills” / knowledge files**, version-controlled in git. One file per concept: `qualified-lead-definition.md`, `pricing-structure.md`, `glossary.md`, `kpi-definitions.md` (e.g., exact ROAS/CAC formulas), `pipeline-milestone-map.md` (mapping AccuLynx milestones → revenue stages).
- **Two-tier retrieval (mirrors how Hermes and Claude Skills work):**
 1. **Always-on core memory** — small, authoritative definitions (what counts as a qualified lead; the canonical KPI formulas) injected into the system prompt at session start so there is zero retrieval latency and zero chance of a miss.
 2. **On-demand vector retrieval** — larger corpora (full price books, historical SOPs) embedded into a vector DB (pgvector in the same Postgres, or a dedicated store) and exposed as an MCP retrieval tool the agent queries only when needed.
- **Anti-hallucination patterns:** (1) inject definitions, never let the model infer them; (2) require sub-agents to cite the source file/field for any business-logic claim; (3) for any metric (ROAS, CAC, “qualified lead count”), the agent must compute from retrieved raw figures + the canonical formula file, not estimate; (4) keep the “qualified lead” definition identical across the Marketing agent (CallRail `lead_status` /GHL stage) and Sales agent (AccuLynx milestone) so the two never diverge.

Multi-Agent Routing Strategy (worked example)

Query: "How is Victor's pipeline looking compared to our Google Ads spend this week?"

1. **Channel → Orchestrator.** Slack Bolt (Socket Mode) receives the message, verifies the signing secret, strips/escapes the raw text, and hands a structured request to Jarvis.
2. **Query parsing & decomposition.** Jarvis identifies two domains (Sales: "Victor's pipeline"; Marketing: "Google Ads spend") and one time window ("this week" → resolves to an explicit ISO date range). It resolves "Victor" against the AccuLynx `GET /users` list to a user GUID (and confirms one match to avoid ambiguity).
3. **Parallel dispatch (Task tool).**
 - **Sales sub-agent** calls `mcp__acculynx__get_jobs` filtered by `salesOwner=<Victor-GUID>`, date range, and milestone, then `get_payments_overview / get_financials` to total open vs. won pipeline value. Read-only GET tools only.
 - **Marketing sub-agent** calls `mcp__googleads__search` with a GAQL query (`SELECT campaign.name, metrics.cost_micros, metrics.conversions_value FROM campaign WHERE segments.date DURING ...`), converts `cost_micros ÷ 1e6`.
4. **Sub-Agents → Data sources.** Each MCP server injects its own credential (AccuLynx Bearer key; Google Ads OAuth access token, refreshing if expired), enforces the read-scope, applies backoff on 429s, and returns structured JSON to its sub-agent.
5. **Synthesis.** Sub-agents return compact summaries to Jarvis, which reconciles the time windows, computes the comparison (e.g., pipeline value created vs. ad spend → a blended ROAS proxy using the canonical formula from `kpi-definitions.md`), and writes one Slack-formatted answer with the underlying numbers and their sources.
6. **Channel ← Orchestrator.** Jarvis posts the synthesized answer back to the Slack thread.

Step-by-Step Implementation Roadmap

Phase 0 — Foundations (week 1). Provision the VPS; harden (non-root user, firewall, Tailscale, disk encryption); install the Claude Agent SDK runtime; stand up a private git repo for Skills/knowledge files; create a secrets vault (env vars or a managed secrets store — never hardcode). **Submit the Google Ads developer-token application on day one** (longest external dependency).

Phase 1 — Read-only Reporting MVP (weeks 2–3). Build the Postgres MCP server with one read-only `query` tool against a `GRANT SELECT -only` role on a read replica. Wire Slack Bolt in Socket Mode. Ship a single-agent Jarvis that answers questions against the ROI dashboard only. **This is the smallest safe, useful product.** Success benchmark: 95%+ correct answers on a 20-question revenue-target test set with zero write attempts reaching the DB.

Phase 2 — Marketing+Sales combined agent (weeks 4–7). Build MCP servers for Google Ads (after securing the approved developer token), GHL v2, AccuLynx, and CallRail — each exposing GET/read tools only. Add the combined Marketing+Sales sub-agent. Author the Layer 6 knowledge files. Success benchmark: the worked-example query above returns correct, sourced numbers; cross-funnel attribution (GCLID→job) resolves end-to-end.

Phase 3 — Hardening & memory (weeks 8–9). Add persistent memory, the two-tier vector retrieval, prompt-injection guardrails (Layer 5), per-tool audit logging, and rate-limit/backoff handling. Load-test against AccuLynx (10 req/s/key) and GHL (100 req/10s) limits.

Phase 4 — Scale the topology (as triggers fire). Split Marketing and Sales into separate sub-agents when the combined tool list or routing accuracy degrades. Add webhook ingestion (AccuLynx milestone/invoice topics, GHL events, CallRail post-call) to move from polling to event-driven freshness.

Phase 5 — Voice (optional, later). Attach Vapi+ElevenLabs at the transport edge; reuse the identical orchestrator/sub-agent core.

Security & Guardrail Protocol

Enforcing read-only (defense in depth — never rely on the prompt alone):

1. **Database layer:** dedicated Postgres role with only `GRANT SELECT` ; no INSERT/UPDATE/DELETE/DDI; point at a read replica; `set statement_timeout` and a row-limit; consider query rewriting to restrict accessible tables/columns.
2. **Tool layer:** MCP servers for AccuLynx/GHL/Google Ads/CallRail implement **only GET/read tools** — write endpoints are simply never coded, so the model cannot call them regardless of what it “decides.” Use the SDK `allowedTools allowlist` (e.g., `mcp__acculynx__get_*`) rather than broad permissions; prefer explicit allowlists over `bypassPermissions` .
3. **Prompt layer:** system-prompt instruction that the agent is strictly analytical/read-only, reinforced by an LLM-as-judge or deterministic input screen.

Credential masking & secrets:

- Store all keys/tokens in a secrets vault or env vars injected at runtime; never in the repo or in CLAUDE.md. Add `.env` to `.gitignore` .
- Each MCP server owns exactly one platform’s credential; the orchestrator holds none. This limits blast radius if any single sub-agent context is compromised.
- Rotate OAuth refresh tokens; handle Google Ads / GHL 24-hour access-token expiry automatically; store the AccuLynx per-location keys encrypted at rest.
- Never echo credentials or full DB connection strings into model context or logs.

Preventing prompt injection from corrupting API payloads:

- **Separate control from data:** treat all retrieved content (call transcripts, CRM notes, lead names) as untrusted data, never as instructions. Structured/parameterized tool calls (analogous to SQL prepared statements) keep model-generated query parameters from being interpreted as commands.
- **Input screening + output validation:** validate/normalize user input (length, encoding) before it reaches the orchestrator; validate every model-proposed tool argument against a schema before execution. Published research on MCP-schema “contract” engines reports blocking ~98% of unsafe queries (e.g., write attempts in a read-only context) at ~8–13 ms overhead per tool call — negligible against LLM generation time.
- **Defense in depth:** assume injection will eventually succeed and rely on least-privilege tool scopes and read-only roles to bound the damage — a compromised read-only agent can only read non-destructive data. Optionally layer a guardrail model (Llama Guard / NeMo Guardrails), remembering the guardrail LLM is itself injectable and is one layer, not the whole defense.
- **Audit everything:** log every tool call (who/what/when/which params) for after-the-fact review.

Recommendations

1. **Adopt Claude Agent SDK on a Hetzner/DigitalOcean VPS now; defer the Mac Mini** unless iMessage delivery is a hard requirement from the principals.
2. **Kill the “AccuLynx is impossible” assumption** — it has a documented read API. Apply for an AccuLynx Partner Program slot early to raise rate limits and get a (limited) support channel.
3. **Apply for the Google Ads developer token in week 1** — Google’s official review target is ~2 business days, but an acknowledged early-2026 backlog can stretch it to weeks; this is the longest external dependency on the critical path.
4. **Ship the read-only Postgres Reporting agent as the MVP** (Phase 1) — lowest risk, immediate value, proves the guardrail stack before any other system is touched.
5. **Build every connector as a GET-only MCP server** — this is the cheapest, most reliable way to enforce read-only across heterogeneous APIs.
6. **Budget for CallRail’s API usage tier** (\$80/mo Premium up to 12,000 calls, or \$200/mo Platinum up to 300,000) and **Premium Conversation Intelligence** if you want transcript/sentiment/lead-score signals.
7. **Standardize the “qualified lead” definition in one Markdown file** referenced identically by the Marketing and Sales agents so the two never report divergent numbers.

Thresholds that change the plan: move off Replit the moment you need persistent OS-level browser drivers; split to 3 agents when the combined agent exceeds ~20–25 tools or routing accuracy drops; switch the harness to LangGraph if you require model-agnosticism or strict deterministic/auditable cyclic workflows; introduce the Mac Mini only when iMessage or local screen automation becomes mandatory.

Caveats

- **Source quality:** Google Ads, GHL, AccuLynx, CallRail, Slack, Anthropic, and Replit facts are drawn from official/primary documentation (several confirmed verbatim against vendor docs). Framework-comparison and hosting claims (e.g., LangGraph “100M+ runs/month,” Mac-vs-VPS break-even ranges, Vapi per-minute pricing) come from vendor blogs and should be treated as directional, not audited.
- **Pricing and rate limits drift.** All dollar figures and limits (CallRail tiers, GHL 100/10s, AccuLynx 10/s) were current as of mid-2026; re-verify against live docs before committing budget.
- **AccuLynx field-level depth is unverified for Shumaker’s specific account.** The endpoint catalog is confirmed public, but actual data exposure depends on Shumaker’s AccuLynx plan tier and whether custom workflows are enabled (some milestone/status fields require “custom workflows” turned on).
- **The read-only phase intentionally excludes** all operations/production/scheduling per scope; any future write capability (e.g., updating a lead) would require re-running this entire guardrail analysis.
- **Hermes Agent’s self-improving memory** is attractive for the channel layer but its always-accumulating memory is an injection surface; do not let it write business-logic memory unsupervised in a finance-adjacent system.